

KavachX

Graph-grounded autonomous cyber-reasoning with proof-carrying repair. It finds a vulnerability, shields the system in minutes, repairs the root cause, attacks its own repair, and issues a signed certificate in which every claim drills down to executable evidence.

KAVACHX — THE SYSTEM

SAMHITA — THE BEHAVIOURAL CONTRACT

PRAMAAN — THE PROOF

01

Executive summary

KavachX answers the AI Kavach problem statement — *"build a cyber-reasoning system that autonomously finds a vulnerability, patches it, and proves the fix holds"* — by inverting the pipeline everybody else will build.

The conventional design is **bug-first**: fuzz until something crashes, ask a model for a patch, re-run the crash to confirm. That design is now a commodity — all seven finalists of DARPA's AI Cyber Challenge open-sourced systems with exactly that shape — and its verification step is measurably unreliable.

KavachX is **specification-first**. Before hunting for anything, it reconstructs the specification the target software never had, as an executable artifact called **SAMHITA**. Once that exists, all three clauses of the problem statement become operations on one object:

- **Vulnerability** — a reproducible counterexample to a contract clause.
- **Patch** — the minimal change that removes the counterexample.
- **Proof** — the counterexample is gone *and every other clause still holds*.

That third line quietly solves something nobody else will notice. The problem statement demands a *"regression test harness"* — but bespoke military software has no test suite to run. In KavachX the harness is synthesised, not assumed: **SAMHITA is the test suite the target never shipped with.**

Two deliverables are described in this document. The **product PoC** (§9–§16) is a web service: a user submits a repository URL, and KavachX indexes, analyses, exploits, repairs, verifies and opens a pull request — autonomously, inside a hardened sandbox, under role-based control. The **finale system** (§5–§8) generalises the same engine to targets with no repository at all.

Competition context and compliance

PARAMETER	CONFIRMED VALUE
Problem statement	"Build a cyber-reasoning system — an LLM laced with fuzzers, static and dynamic analysis, and a regression test harness — that autonomously finds a vulnerability, patches it, and proves the fix holds."
Shortlisting criteria	Resource utilisation · novelty of idea · how light-weight the solution is
Finale criteria	Performance · speed · precision · functionality · scalability
Finale weighting	Innovation & Relevance 20% · Feasibility 20% · Illustration of Idea 20% · Technical Depth 20% · Presentation & Demonstration 20%
Submission	Max 5 slides — intro/ideation · methodology · stack & flow · novelty/USP · deliverables & PoC
Team	Max 3 including leader · Indian citizens · single track
Key dates	Registration to 20 Aug 2026 · Shortlisting 1–10 Sep · Finale 6–8 Oct (36 h, New Delhi) · Awards 9 Oct
Target environment	"Pitched to run autonomously against specific customised infrastructure of the Indian Armed Forces"

Requirement coverage

NAMED REQUIREMENT	KAVACHX COMPONENT
LLM	Six schema-constrained call types, each gated by a deterministic check (§7)
Fuzzers	AFL++ / libFuzzer with QEMU mode; Harness Foundry synthesises the driver (§6)
Static analysis	GitNexus code graph · Semgrep · deployment graph
Dynamic analysis	Sanitizers, ptrace/Frida instrumentation, runtime monitors, recorded traces
Regression test harness	SAMHITA — synthesised, because the target has none
Finds a vulnerability	Four parallel discovery channels → hypothesis queue → executable validation
Patches it	Root-cause repair, plus a reversible shield on the fast path
Proves the fix holds	PRAMAAN — refutation gauntlet, graded assurance, evidence graph
Resource utilisation	Two local models, MoE workhorse, live token meter
Light-weight	One laptop, ≤16 GB, zero network egress, single embedded database
Unfamiliar infrastructure	Three Probe adapters — repository, bare binary, or a network port

Why the obvious design loses

Three findings from the research shaped every decision in this document.

FINDING 1 – THE CONVENTIONAL ARCHITECTURE IS A COMMODITY

DARPA's AI Cyber Challenge concluded in August 2025. All seven finalist cyber-reasoning systems were open-sourced, and the systematisation paper on the competition finds they **converged on a single architecture**: dual proof-of-vulnerability pipelines feeding RCA → Generate → Validate → Dedup → Submit. That pipeline is the problem statement, and it is downloadable. Proposing it scores zero on novelty.

FINDING 2 – VERIFICATION IS WHERE THE FIELD ACTUALLY FAILS

Patches that built cleanly *and* stopped the crashing input were still **semantically wrong 37–46% of the time** — wrong root cause, symptom-level patching, silent behavioural changes, new bugs introduced. "Re-run the exploit and run the tests" is an empirical coin flip, and it is exactly what most competing teams will demonstrate.

FINDING 3 – THE RUBRIC INVERTS THE STATE OF THE ART

Each AlxCC finalist was provisioned with roughly **\$85,000 of compute and \$50,000 of LLM credits** for a 143-hour run. Al Kavach shortlists on resource utilisation and how light-weight the solution is. The state of the art is disqualified by the scoring sheet; that gap is the opening.

A fourth, structural fact governs the finale. Every AlxCC system assumed full source, a standard build system, a pre-written fuzzing harness and an existing test suite — the paper is explicit that binary-only and closed-source scenarios were never addressed. In 2025 the equivalent Terrier track ran its live hunt on **BOSS Linux**; the Ministry of Defence also fields **Maya OS**. Bespoke services on a bespoke distribution.

The bottleneck at the finale is not analysis. It is bootstrapping — manufacturing a harness, an oracle and a specification for software nobody has ever fuzzed.

04

Core concept

SAMHITA — the behavioural contract

Samhitā: a systematically arranged, authoritative collection of rules. KavachX observes the target under benign workload, records value profiles at every observable boundary, has the language model **propose** candidate rules as executable predicates, then runs a deterministic falsifier that **deletes every rule that fails** against held-out traces. What survives is the contract.

A typical contract holds 30–60 clauses: *this field is never longer than 255 bytes; this function never invokes a shell; this response is deterministic for a fixed input; this counter is monotonic*. Hallucinated clauses die in milliseconds and never reach a human.

The contract then does three jobs: it **directs the fuzzer** (falsify clause 17 rather than chase raw coverage), it **defines a finding** (a violation, not merely a crash — which catches non-crashing logic flaws), and it **serves as the regression harness** after a patch.

PRAMAAN — the proof

Pramāṇa: a means of valid knowledge; evidence. PRAMAAN is an evidence graph plus a signed certificate. Every claim in the certificate carries provenance edges to the artifact that established it.

```
Vulnerability V17
├── discovered by → channel: graph/static, query #4
├── clause violated → SAMHITA #17 (header length bound)
├── code evidence → hdr.c:340 (root cause)
├── graph evidence → 14 callers via impact analysis
├── runtime evidence → trace #9231, sanitizer report
├── exploit evidence → PoV #44 (sha256:1f0c...) reproduced 10/10
├── shield → filter rule #3, deployed t+3m20s, reversible
├── patch → diff (sha256:9c4e...), iteration 2 of 3
└── verification → gauntlet 4/4 · replay 5000/5000 · clauses 40/40
```

Assurance is **graded, never boolean**. This is a deliberate defence: fuzzing and replay establish bounded empirical assurance, not formal proof, and a rigorous evaluator will dismantle any team that claims otherwise.

LEVEL	EARNED WHEN
A	Exploit eliminated · all clauses hold · differential replay byte-identical over the full benign corpus · survived mutation and sibling hunt · coverage delta confined to the guarded branch
B	Exploit eliminated · clauses hold · replay clean · sibling hunt surfaced unproved candidates elsewhere
C	Exploit eliminated · one or more clauses could not be re-verified — behaviour change detected and reported, not concealed
R	Refuted. Patch withdrawn, shield remains deployed, refuting input attached

Shield-first — two clocks

Time-to-protection and time-to-repair are separated and optimised independently. On the fast path, a reversible mitigation — a seccomp filter, an input-filter rule, an `LD_PRELOAD` guard — is derived from the exploit and its syscall trace, verified against two deterministic checks (the exploit is blocked; the recorded benign corpus still passes), and deployed in minutes. The certified source patch follows on its own schedule.

This is not decoration. A fielded system cannot be recompiled mid-operation, and the track is named *Kavach* — armour.

05

System architecture

Seven planes. Discovery runs as parallel channels so that no single component failure produces an empty result, and the main graph is a loop over a work queue rather than a single pass — which makes output **anytime**: stop the system at any moment and completed certificates are already available.

PLANE 0 – CONTROL

Super Agent (thin router: picks phase and budget, never decides verdicts)
Budget Governor · Checkpointer · Failure Ledger

reads priors at START → CYBER IMMUNE MEMORY
(clause templates, bug-class rules)

PLANE 1 – SUBSTRATE

PROBE – classifies the target, selects ONE adapter:
[A] source repository [B] opaque ELF / process [C] network service
→ emits INTERFACE DESCRIPTOR (entry points, input shape, invocation)
SANDBOX = execution context for everything below
EVIDENCE STORE = one database. Everything is a row. Resumable.

PLANE 2 – UNDERSTANDING

(builds the WORLD MODEL, continuously)

REPOSITORY INTELLIGENCE

GitNexus – adapter A
ACCELERATOR, optional
call graph · impact ·
attack surface · trace
handles, not contents

DEPLOYMENT GRAPH

units, ports,
permissions,
principals,
reachability
from untrusted

SAMHITA SYNTHESIS ★

observe benign runs
→ LLM PROPOSES rules
→ FALSIFIER destroys
every false one
→ survivors = SPEC

PLANE 3 – DISCOVERY

(four PARALLEL workers; none gates another)

ch1 GRAPH / STATIC
taint and sink
queries; first
output at t+90s

ch2 CONFIG REACH
privilege and
exposure facts

ch3 FUZZ

ch4 RUNTIME MONITOR
sanitizers,
trace anomalies

HARNESS FOUNDRY (async)
create → smoke → repair ≤3
GATE: coverage must MOVE

HYPOTHESIS QUEUE
priority = reachability
× confidence × blast

← all channels push
main loop pops

PLANE 4 – EVIDENCE

VALIDATOR – converts a hypothesis into an EXECUTABLE exploit, runs it

confirmed → CORRELATE → ATTACK GRAPH → PRIORITIZER
dedup across chains "closes N paths"
unconfirmed → DOWNGRADE + LEDGER (never silently discarded)

PLANE 5 – RESPONSE

(two paths, two clocks)

FAST ▶ SHIELD SYNTHESIS ▶ verify: exploit blocked ∧ benign passes ▶ DEPLOY
seccomp / input filter / LD_PRELOAD · REVERSIBLE · minutes
SLOW ▶ CAUSAL ANALYST ▶ PATCH SYNTHESIS ▶ BLAST RADIUS ▶ APPLY TO A COPY
reverse execution root cause, ← impact analysis defines
to FIRST CAUSE not symptom the REGRESSION SCOPE

REFUTATION GAUNTLET – four stages, in PARALLEL

① exploit mutation ② sibling hunt
(bypass the patch) (find-one, find-all)
③ differential replay ④ SAMHITA re-check on
over benign corpus affected callers only

FAIL

refutation becomes a
CONSTRAINT and a new
clause → PATCH
(max 3 rounds, then
HONEST FAILURE;
shield stays up)

PASS

PLANE 6 – ATTESTATION
PRAMAAN evidence graph
▶ CERTIFICATE A/B/C/R
▶ MEMORY written back
▶ RESIDUAL-RISK MAP

Component register

Eleven **P0** components form the thin thread. They must run end-to-end before anything else is written.

#	COMPONENT	LLM	MODEL	BUILD
1	Probe / target classifier	no	—	P0
2	Interface hypothesis	yes	workhorse	P0
3	Deployment graph	no	—	P0
4	SAMHITA synthesis	yes	workhorse	P0
5	Clause falsifier	no	—	P0
6	Graph / static channel	yes	router	P0
7	Hypothesis queue	no	—	P0
8	Validator (exploit execution)	no	—	P0
9	Patch synthesis	yes	workhorse	P0
10	Refutation gauntlet ×4	no	—	P0
11	PRAMAAN attester	no	—	P0
12	Repository intelligence (GitNexus)	thin	router	P1
13	Harness foundry + fuzz channel	yes	workhorse	P1
14	Causal analyst (reverse execution)	yes	workhorse	P1
15	Shield synthesis	yes	workhorse	P1
16	Blast-radius review	no	—	P1
17	Correlation	no	—	P2
18	Attack graph + prioritizer	no	—	P2
19	Cyber immune memory	no	—	P2

Division of authority

The language model is allowed to propose. It is never allowed to decide.

This is the structural defence against the charge that KavachX is "another LLM security agent", and it is verifiable straight from the architecture. Every LLM call has a defined input, a schema-validated output, and a deterministic check that can reject it.

CALL	INPUT	OUTPUT	CHECKED BY
Interface hypothesis	ELF exports, strings, syscall summary, unit file, open ports, sample packets	Entry points and input shapes, as JSON	Harness smoke test — coverage must move
Clause proposal	Boundary signature plus observed value profiles (min, max, null rate, type, cardinality) and three sample traces	Executable predicates in the clause DSL	Falsifier over held-out traces
Constraint authoring	Blocked branch condition from source or decompilation	SMT constraint over input bytes	Solver produces an input; the branch must actually be reached
Root-cause hypothesis	Causal chain from reverse execution — last writer, allocation site, taint path	Ranked hypothesis naming file and line	A patch at that site must survive the gauntlet
Patch authoring	Root cause, the contract clauses touching that function, minimal code slice	Minimal unified diff	Build, exploit replay, differential replay, contract re-check
Refutation strategy	Candidate diff and the original exploit	Bypass <i>strategies</i> , not payloads	Executed by the fuzzer; only real bypasses count

Never delegated to a model

- Whether a crash occurred — the sanitizer and exit status decide.
- Whether a contract clause holds — the evaluator decides.
- Whether a patch is correct — differential replay and the contract decide.
- Deduplication — stack hash plus coverage signature decides.
- Severity and reachability — the graph decides.
- Any measurement: coverage, tokens, timing, false-positive rate.
- **Anything that appears in PRAMAAN as evidence.**

08

State model

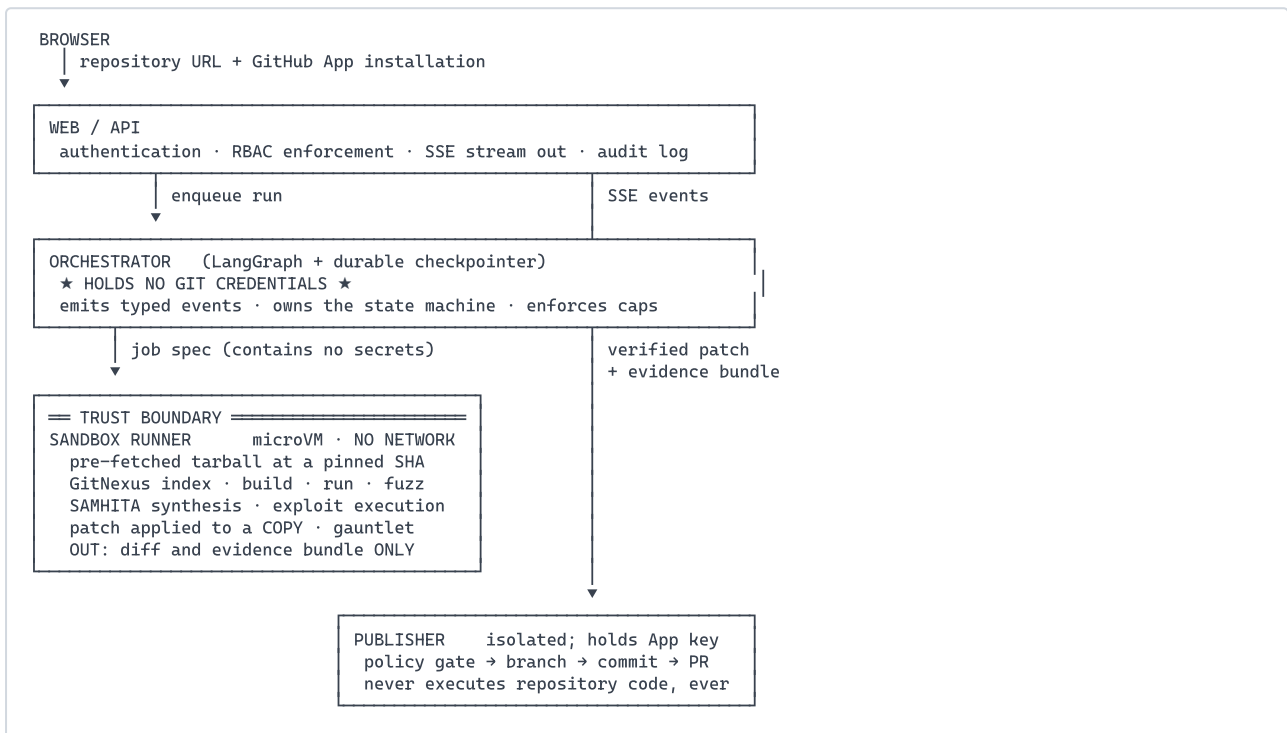
```
class KavachState(TypedDict):
    run_id: str; phase: str
    target: dict          # kind: repo | binary | service + interface descriptor
    world: dict           # GRAPH HANDLES, not graph contents (anti-context-explosion)
    samhita: list         # [{clause_id, predicate, scope, obs_n, status}]
    benign_corpus_ref: str # the regression oracle the target never shipped
    hypotheses: list; validated: list; downgraded: list
    attack_graph: dict; priority: list
    shields: list         # {id, rule, revert_cmd, verified_blocked, verified_benign}
    patches: list         # {diff, root_cause, blast_radius[], iter}
    gauntlet: dict        # per-stage verdicts
    pramaan: dict         # claim -> [evidence node ids]
    certificates: list    # {level: A|B|C|R, evidence_hashes[]}
    ledger: list          # every stall, refutation, unreached branch, with cause
    budget: dict          # tokens spent, calls, yield per node
    iter: dict            # caps: harness <= 3, patch <= 3, clause <= 2
```

Two decisions carry disproportionate weight. `world` stores **handles, not contents** — agents ask the graph targeted questions rather than loading a repository into a context window, which is both the correctness fix and the resource-utilisation story. And every field is persisted as database rows after each node, so a run that dies at hour three **resumes** rather than restarting — the AlxCC survival lesson made structural.

The product PoC

A user signs in, installs the KavachX GitHub App on a repository, and submits its URL. Everything after that is autonomous: index, understand, discover, validate, repair, verify, document, and open a pull request. The browser shows the system reasoning in real time.

Because the input is a repository, Probe adapter A is the only one required — which makes GitNexus legitimately load-bearing for the PoC. Adapters B and C are finale-hardening work (\$19).



The process that runs untrusted code and the process that holds credentials are different services, with no shared filesystem and no shared network.

The runner's only output is a diff plus an evidence bundle — inert data. This single separation is what makes it safe to point the system at a stranger's repository.

Three threats most designs miss

THREAT 1 – UNTRUSTED CODE EXECUTES BEFORE ANY FUZZING BEGINS

The moment the system runs `pip install -e .`, `npm install` or `make`, it has executed arbitrary code from a stranger's repository. `setup.py`, `postinstall hooks`, `Makefile targets`, `conftest.py` and `.envrc` are all remote code execution on the build host. **Isolation must be live from ingest, not from the analysis stage.**

THREAT 2 – A CREDENTIAL IN THE SANDBOX COMPROMISES EVERY CONNECTED REPOSITORY

If the environment that executes untrusted code also holds the token that can push, a single malicious repository owns every repository the application can reach. Hence the trust boundary in §9 — the runner holds no secrets of any kind, including environment variables and metadata endpoints.

THREAT 3 – A PATCH TOUCHING CI CONFIGURATION IS A SECRET-EXFILTRATION PRIMITIVE

An autonomous agent able to modify `.github/workflows/` in a pull request can, on merge or via `pull_request_target`, print repository secrets. This must be a **hard-denied path in the policy gate**, not a lint warning.

11 Sandbox specification

CONTROL	SETTING
Isolation	microVM (Firecracker or gVisor). A plain container is not sufficient for arbitrary code execution.
Network	None . No network namespace. Dependencies come from a pre-populated offline mirror mounted read-only.
Filesystem	Read-only root; one writable tmpfs scratch; no host mounts
Identity	Non-root; <code>no-new-privs</code> ; all capabilities dropped; seccomp profile
Limits	cgroups — CPU quota, memory cap, PID cap, disk quota, wall-clock kill
Secrets	Zero . No environment variables, no token files, no metadata endpoint
Source delivery	Pre-fetched tarball at a pinned commit SHA — never <code>git clone</code> inside the sandbox, which removes credential helpers and submodule fetches as attack surface
Egress	Structured artifacts only, through a single serialisation channel

Two profiles share the same isolation but differ in quota: **analysis** (indexing, static queries, contract synthesis — short and cheap) and **execution** (build, fuzz, exploit, gauntlet — long, expensive, hardest limits).

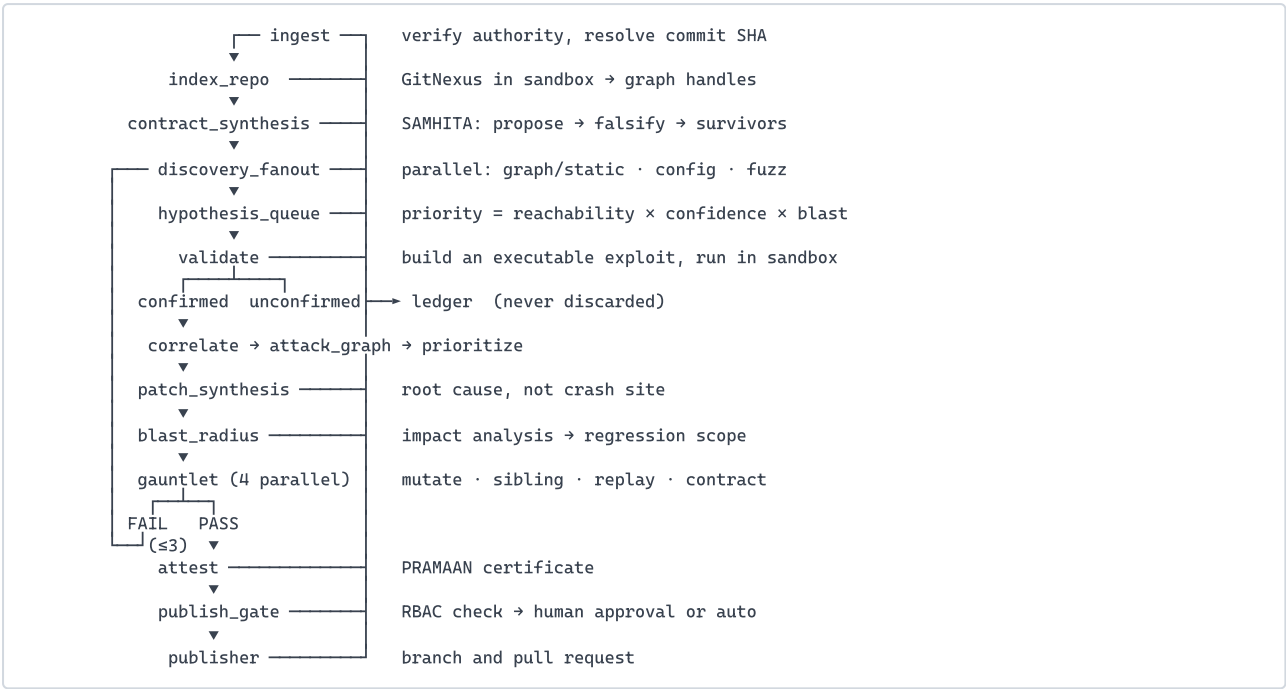
Prompt injection

Repository content is **data, never instruction**. Every model call receives structured fields and returns schema-validated output; there is no "here is the file, do what it says" path. A README containing *ignore previous instructions* reaches the model as a string field, and the model's output is still gated by a deterministic check before it can affect anything.

12 LangGraph orchestration

LangGraph is used as a **thin state machine**. Long-running work — indexing, fuzzing, replay — runs as sandbox jobs the graph waits on, never as graph nodes. A six-hour fuzzing campaign modelled as a node

is the mistake that consumes an entire build day.



Routing is always deterministic. The Super Agent selects phase and budget; it never decides confirmed, refuted, or publish.

```
def route_after_gauntlet(s):
    if all(s["gauntlet"][k] == "pass" for k in
          ("mutation", "sibling", "replay", "contract")):
        return "attest"
    if s["patch_iter"] >= 3:
        return "record_honest_failure" # shield remains deployed
    return "patch_synthesis"          # refutation becomes a constraint
```

State is checkpointed after every node. Iteration caps are hard: harness ≤ 3, patch ≤ 3, clause ≤ 2. There is no unbounded loop anywhere in the system.

13 Patch to pull-request pipeline

Policy gate

Deterministic, executed inside the Publisher, before anything is pushed. Only patches carrying a level A or B certificate reach it at all.

RULE	ACTION ON VIOLATION
Path denylist — <code>.github/**</code> , Dockerfiles, CI YAML, <code>.git*</code> , lockfiles	Hard reject
New dependency introduced	Hard reject (v0)
New network call, or new <code>exec</code> / <code>eval</code> / <code>subprocess</code>	Hard reject
Binary file modified	Hard reject
Certificate absent, or level R	Hard reject
Diff exceeds line or file cap	Reject; request a narrower patch

RULE	ACTION ON VIOLATION
Touches files outside the computed blast radius	Reject

Conflict-aware branch layout

KavachX computes the file-overlap graph across verified patches and selects the layout automatically. This is the difference between a tool that produces six conflicting branches and one a maintainer can actually merge.

Disjoint fixes → **parallel pull requests**, each based on the default branch and independently mergeable:

```

main ┌ kavachx/run-7f3a/fix-01-heap-overflow → PR #101 → main
    │ kavachx/run-7f3a/fix-02-path-traversal → PR #102 → main
    └ kavachx/run-7f3a/fix-03-integer-overflow → PR #103 → main

```

Overlapping fixes → **stacked pull requests**, each targeting the one below, so no conflicts arise:

```

main — fix-01-auth-bypass — fix-02-session-fixation — fix-03-idor
      PR #104 → main      PR #105 → fix-01      PR #106 → fix-02

```

An umbrella pull request from the top of the stack to the default branch carries the run summary and the three deliverable documents.

Guarantees

- Never pushes to a default or protected branch.
- Never force-pushes, amends, or rewrites history.
- One commit per fix, signed, with a structured trailer.
- The pull-request body embeds the PRAMAAN certificate — the pitch lands where a reviewer is already looking.

```

Fix: heap buffer overflow in parse_header()

Root cause: the length check at hdr.c:340 uses a signed comparison,
allowing a negative length to bypass the bound at hdr.c:812.

Assurance: PRAMAAN level A
Evidence:  run 7f3a · certificate sha256:9c4e...
Exploit:   reproduced pre-patch 10/10, blocked post-patch 10/10
Replay:    5,000 recorded requests, byte-identical
Contract:  40/40 SAMHITA clauses hold · 14 affected callers verified
Blast:     hdr.c, hdr.h and 14 call sites (impact-scoped)

Co-Authored-By: KavachX <noreply@kavachx.local>

```

Progress streams over Server-Sent Events. **Raw model tokens are never streamed** — they are noisy, they leak prompt internals, and they read as less capable. What streams instead is structured reasoning, which is both safer and more persuasive.

```

type RunEvent =
| { t:"phase";   phase:string; status:"start"|"done"|"failed" }
| { t:"thought"; agent:string; hypothesis:string; evidence:string[];
  decision:string; confidence:number }
| { t:"tool";    name:string; target:string; ms:number; ok:boolean }
| { t:"finding"; id:string; state:"hypothesis"|"validated"|"refuted";
  clause?:string; severity:string; reachable:boolean }
| { t:"diff";    finding:string; file:string; patch:string; iter:number }
| { t:"gauntlet"; finding:string; stage:string; verdict:"pass"|"fail";
  detail:string }
| { t:"metric";  tokens:number; coverage:number; ram_mb:number; egress:0 }
| { t:"artifact"; kind:"certificate"|"pr"|"docs"; url:string };

```

Five panels

Pipeline timeline

Phases with live status; the spine of the run.

Reasoning trace

The `thought` stream — hypothesis → evidence → decision. This is what a thinking pattern should actually look like.

Findings table

State chips: hypothesis, validated, refuted. Severity and reachability encoded in form, not only number.

Diff viewer

Each patch beside its gauntlet verdict and iteration number.

Resource meter

Tokens, coverage, memory — and **network egress pinned at zero**.

The moment to design for

A `gauntlet` event with `verdict:"fail"` must render loudly. The system rejecting its own patch is the most persuasive thing on the screen.

15

Deliverable documents

Three artifacts, committed to the umbrella pull request and downloadable from the run page.

CHANGES DONE

CHANGES.md

Per fix — what, root cause rather than symptom, the violated clause, evidence links, assurance level, blast radius, files touched, PR link.

WHAT IS LEFT

REMAINING.md

Hypotheses that could not be validated and why
· patches generated and **refuted**, with the refuting input
· code never reached, with the reason
· coverage debt by module
· decisions requiring a human.

PROOF

certificate.json

The PRAMAAN evidence graph, plus a rendered HTML view for reading.

REMAINING.md is the residual-risk map, and it is a differentiator rather than an admission. "Here is what I did not examine, and why" is what makes an autonomous security system trustworthy rather than merely confident — and it guarantees a substantive deliverable even on a run that finds nothing.

16

Access control and tenancy

Model: **Organisation** → **Project (repository)** → **Run**, with row-level security and a tenant identifier on every table.

PERMISSION	OWNER	MAINTAINER	SEC REVIEWER	DEVELOPER	VIEWER	AUDITOR
run:start	✓	✓	✓	✓	—	—
finding:read (summary)	✓	✓	✓	✓	✓	✓
finding:read_pov (working exploit)	✓	—	✓	—	—	—
patch:review	✓	✓	✓	✓	—	—
patch:publish (opens PR)	✓	✓	✓	—	—	—
policy:manage	✓	—	✓	—	—	—
member:manage	✓	—	—	—	—	—
audit:read	✓	—	✓	—	—	✓
run:abort	✓	✓	✓	✓	—	—

Exploits are the sensitive artifact

A working exploit for a still-unpatched vulnerability is more dangerous than the finding that describes it.

`finding:read_pov` is gated separately and exploits are redacted to a hash plus description for every other role. Most tools treat all findings as one permission level; this one must not.

Autonomous is not unattended

By default the system prepares everything and halts at `publish_gate` awaiting a holder of `patch:publish`. Auto-publish is opt-in per project and available only for level-A certificates. That distinction is exactly what a security-conscious organisation — or a defence evaluator — needs to hear.

Repository authority

Access is granted through a **GitHub App**, not an OAuth personal token. Installation on a specific repository *proves* the user holds authority over it; the app requests fine-grained permissions only (contents: write, pull requests: write) and mints short-lived installation tokens per operation, never persisted and never present in the sandbox. Any repository URL without an installation is refused — which also prevents the service being pointed at repositories the user does not own.

The audit log is append-only and hash-chained. Every event records actor, action, subject, evidence hash and timestamp; publishing a pull request is the highest-value record in the system.

17

Threat model

THREAT	CONTROL
Malicious repository achieves RCE at build time	microVM, no network, no secrets — isolation live from ingest
Credential theft via untrusted code	Credential and execution separation; the runner holds no secrets
Exfiltration during fuzzing or exploitation	Sandbox has no network namespace at all
Prompt injection from repository content	Content is data; schema-validated outputs; deterministic gates
Backdoored patch	Policy gate — no new dependencies, network calls or exec; path denylist; size cap; blast-radius bound
CI configuration tampering	<code>.github/**</code> hard-denied
Scanning repositories the user does not own	GitHub App installation required as proof of authority
Resource exhaustion	cgroups quotas, wall-clock kills, per-tenant concurrency caps
Cross-tenant data leakage	Row-level security, tenant id on every row, per-tenant storage prefixes
Exploit leakage	<code>finding:read_pov</code> gated; exploits encrypted at rest
Runaway autonomous loops	Hard caps: harness ≤ 3 , patch ≤ 3 , clause ≤ 2
Unreviewed autonomous code change	Publish gate with human approval by default

Model portfolio

ROLE	MODEL	Q4 SIZE	RATIONALE
Workhorse harness synthesis, clause proposal, patch authoring	Qwen3-Coder-30B-A3B	18.6 GB	Mixture-of-experts: 30.5 B total but only ~3.3 B active per token — reasons like a 30 B model, runs like a 3 B one. 256 K context. This is the defensible "lightweight" claim.
Router classification, extraction, dedup labels	Qwen3-4B	~3 GB	Ninety per cent of calls are cheap. The large model is never woken for filing.
Security triage OPTIONAL	Foundation-Sec-8B-Reasoning	~5 GB	Security-domain pretrained, permissive licence, built for air-gapped deployment. Cut first if memory is tight.

Ship two models, not three. Each additional model is memory you do not have and a failure mode you do not need. Hardware branches: a 24 GB GPU runs the workhorse directly; an 8–16 GB laptop GPU runs it with expert offload to CPU, which works unusually well for MoE because only 3.3 B is active; a CPU-only 16 GB machine must drop to a 14 B dense model.

DO THIS IN SEPTEMBER, NOT OCTOBER

Take twenty real functions, request clause proposals in strict JSON, and measure schema adherence. Below roughly 85 per cent you need constrained decoding via GBNF grammars — half a day of work you want to discover early rather than at hour six of the finale.

Technology

MUST HAVE

- Python 3.11 · FastAPI · Next.js + SSE
- LangGraph as a thin state machine, durable checkpointer
- Postgres with row-level security
- Firecracker / gVisor sandbox
- GitNexus · tree-sitter · Semgrep
- AFL++ with QEMU mode · ASan / UBSan
- llama.cpp · Z3 · GitHub App client

DO NOT BUILD

- Kubernetes or any orchestrator
- A vector database — everything here has a key
- A fine-tuned model
- A custom sandbox runtime
- Your own fuzzer, decompiler or diff engine
- WebSockets — SSE is sufficient
- A plugin system

Managed sandbox services built on Firecracker are a legitimate option for the PoC and can save several days. Note also that GitNexus ships under a noncommercial licence and its PDG/taint mode

is marked experimental — neither is a problem for the competition, but taint must never be load-bearing, and a self-hosted graph indexer is the substitute if deployment is ever raised.

19

Implementation plan

PoC v0 — three weeks, one language

Pick C *or* Python, not both. Build order matters: **sandbox and authentication before analysis**. Build the clever part first and you will retrofit isolation around code that assumed it had a filesystem and a network — a retrofit that never fully works.

1. Ingest, GitHub App authentication, repository authority check
2. Sandbox runner with the full isolation profile
3. GitNexus indexing in-sandbox, returning graph handles
4. SAMHITA synthesis and the clause falsifier
5. One discovery channel (graph / static) feeding the hypothesis queue
6. Validator producing an executable exploit
7. Patch synthesis → policy gate → gauntlet (replay and contract re-check first)
8. PRAMAAN certificate
9. Publisher: single branch, single pull request
10. SSE interface: timeline, reasoning trace, diff viewer
11. RBAC with three roles and the audit log

DEFINITION OF DONE

Paste a repository URL, watch the system reason, receive a pull request carrying a certificate, `CHANGES.md` and `REMAINING.md`.

PoC v1

Fuzz channel and harness foundry · full six-role RBAC · conflict-aware stacked and parallel pull requests · correlation and attack graph · causal analyst · additional languages · Probe adapters B and C — which is what carries the PoC to the finale system.

Competition timeline

- **to 20 Aug** **Blocking.** Register. Confirm the live problem statement with the organisers. Decide hardware. Benchmark the workhorse for JSON schema adherence.
- **21 Aug – 10 Sep** **The submission.** Build the eleven P0 components against one known-vulnerable target. Slide 5 must carry a **real certificate from a real run** with real numbers, not a mockup — that alone separates you from most of the field at shortlisting.

- **11 Sep – 1 Oct** **Robustness.** Probe adapters B and C — the highest-variance work, do not defer it. Harness foundry, causal analyst with a plain-debugger fallback, shield synthesis. Kill the process at random and confirm it resumes; repeat until boring.
- **2 – 5 Oct** **The offline bundle.** Assume no internet at the venue: model weights, a full package wheelhouse, pre-compiled fuzzers, container images as archives, pinned dependencies. Dry-run on three targets you have never seen. Record the fallback demo video.

The 36 hours

The finale is not when you build. It is when you adapt.

- **0 – 2** Recon only; **write no code.** Classify the target, run the deployment graph for findings within minutes. **Freeze the adapter choice at hour 2.** One person brings the rig up on the practice target in parallel.
- **2 – 6** Foundry on the real target until the smoke test shows coverage moving. Goal: **any** finding, however trivial, carried all the way to a rendered certificate. End-to-end beats deep.
- **6 – 12** SAMHITA synthesis on the real target. Directed campaign. Shield on the first violation. Peak-quality window — spend it on the contract.
- **12 – 18** Patch quality and all four gauntlet stages. **Feature freeze at hour 18.**
- **18 – 24** Long autonomous run. One person owns *only* reliability. Sibling sweep. Deck begins.
- **24 – 30** **Code freeze, no exceptions.** Package evidence, collect metrics, build the demo script, record the fallback video.
- **30 – 36** Rehearse three times on the clock. Sleep in rotation — Presentation carries a full 20 per cent.

Roles for three people: Substrate (probe, foundry, fuzz, sandbox) · Reasoning (SAMHITA, model calls, patching, gauntlet) · Evidence and Delivery (PRAMAAN, metrics, reliability, demo). The third role is the most undervalued and owns 40 per cent of the finale score.

Unique selling propositions

1. **SAMHITA — the system writes the specification the software never had.** The brief demands a regression test harness; bespoke military software has no tests. Every other team will have an empty box there. KavachX synthesises the specification by observation, and that specification becomes the harness.
2. **PRAMAAN — proof-carrying repair with graded assurance.** The output is not "fixed". It is a signed certificate at level A, B, C or R in which every claim drills to executable evidence — attacking a documented 37–46 per cent semantic failure rate.
3. **The Refutation Gauntlet — it must fail to break its own patch.** Four parallel attacks; a rejection returns as a hard constraint and a new contract clause.
4. **Shield-first — time-to-protection is decoupled from time-to-repair.** Reversible mitigation in minutes, certified fix afterwards. A fielded system cannot be recompiled mid-operation.
5. **Sovereign and air-gapped.** Two local models, zero network egress, one laptop, one embedded database — demonstrable by pulling the cable.
6. **Graph-grounded without being graph-dependent.** GitNexus is an accelerator on the bench, not the foundation under the building; adapters B and C carry the run when no repository exists.
7. **Impact-scoped regression.** Impact analysis enumerates every caller of the patched function; those callers' clauses define exactly what must be re-verified. Proof becomes *scoped and complete* rather than best-effort.
8. **Attack-path-aware remediation.** "Fixing this closes three downstream attack paths" beats sorting by severity score, and it speaks the language of avenues of approach.
9. **Safe by construction as a product.** Credential and execution separation, CI-path denial, exploit-level RBAC, human publish gate. Most autonomous repair tools treat security as a deployment concern; here it is the architecture.

Winning features, mapped to criteria

#	FEATURE	CRITERION WON	PROOF ON STAGE
1	Writes its own harness for an unseen binary	Innovation · Functionality	Ninety seconds, live
2	Prints a 40-clause SAMHITA contract	Technical Depth · Illustration	An artifact nobody else has
3	A finding is "clause 17 violated", not "it crashed"	Precision	Named rule plus the input
4	Shield deployed at 3 min 20 s	Speed · Relevance	Timer visible
5	Rejects its own patch, on stage	Presentation · Precision	The moment they remember
6	5,000-request differential replay, byte-identical	Technical Depth	No test suite required
7	Signed certificate handed over on paper	Illustration · Feasibility	A physical object

#	FEATURE	CRITERION WON	PROOF ON STAGE
8	Find-one, find-all sibling sweep	Scalability	One finding becomes four in ten seconds
9	Token and memory meter · network bytes zero	Resource utilisation · Lightweight	Cable on the table
10	Residual-risk map of what was <i>not</i> examined	Relevance · Feasibility	Honesty reads as maturity

22

Risk register

RISK	LIKELIHOOD	MITIGATION
Finale target outside all three adapters (Windows, web, firmware)	High	Keep adapter C generic — anything that answers a socket. Demo on a stand-in while the real target runs behind.
Scope collapse — three people, nineteen components	High	The eleven P0 components only, until they run end-to-end.
Nothing runs on the day	Medium	Hour-24 freeze, checkpoint and resume, recorded fallback video.
System finds no vulnerability live	Medium	SAMHITA, PRAMAAN and the residual-risk map are the stars; the bug is the occasion. Carry a seeded backstop and disclose that it is seeded.
Judged as "another LLM agent"	Medium	Lead with artifacts, never draw an agent-swarm diagram, state the propose/decide rule aloud.
Overclaiming the word "proof"	Medium	Say <i>bounded empirical assurance, graded and evidenced</i> . Never bare "proof".
Local model too weak for strict JSON	Medium	Measure in September; GBNF constrained decoding as the remedy.
Reverse execution refuses to run in the venue VM	Medium	Plain-debugger watchpoint fallback, tested in advance.
GitNexus PDG mode is experimental	Medium	Never let taint be load-bearing; the graph layer stays P1 enrichment.
Noncommercial licence on a dependency	Low-Med	Acceptable for the competition; flag it, and keep a self-hosted indexer as the swap.

23

Demonstration

Eight minutes. The network cable comes out and stays visible on the table.

0:00	An unfamiliar service — one component with source, one an opaque binary. No tests, no harness, no internet.
0:30	The Unknowns Board: the system declares what it does not yet know. Beginning with uncertainty makes everything after it credible.
1:00	It writes its own harness. Coverage starts moving. This is where a scanner would have stopped.
2:00	SAMHITA appears — forty clauses. "This is the specification nobody wrote. It is also our regression harness."
3:00	Clause 17 violated , with the exact input that broke it — not merely "a crash".
3:20	Shield deployed. Exploit blocked; benign traffic still passing.
4:00	"It crashed at line 812. The bug is at line 340." The causal chain, not a stack trace.
4:30	Patch v1 generated at the root cause.
5:00	The system rejects its own patch. Verdict R, in red. Say nothing for three seconds and let them read it.
5:45	Patch v2 survives all four gauntlet stages.
6:15	5,000 recorded requests replayed, byte-identical. 40 of 40 clauses hold. Impact analysis names 14 affected callers; all verified.
7:00	The PRAMAAN certificate is printed and handed to the judges.
7:45	Sibling sweep: one finding becomes four, each independently proved.
8:15	Network bytes: zero. Against \$135,000 per team at the state of the art.

24 The five-slide submission

SLIDE	WHAT IT MUST CARRY
1 • Introduction	Open on the gap, not the solution: the state of the art is open source, requires source plus a harness plus tests plus a hyperscaler, and therefore cannot run on Armed Forces infrastructure. Then the inversion — build the specification first. Name KavachX, SAMHITA and PRAMAAN, and say what the words mean.
2 • Methodology	The loop, plus the table of six model calls each with its deterministic check. That table is what separates this from every agent-swarm submission in the room.

SLIDE	WHAT IT MUST CARRY
3 · Stack and flow	Seven planes, three Probe adapters, the offline model path. Deliberately boring, established tools. This is the Feasibility slide in disguise.
4 · Novelty and USP	PRAMAAN with the A/B/C/R assurance table and the 37–46 per cent figure it exists to attack. Shield-first. The air gap. Name explicitly what you are <i>not</i> building.
5 · Deliverables and PoC	A real certificate from a real run, a real SAMHITA excerpt, and the numbers: one laptop, zero egress, tokens per finding, time-to-protection against time-to-repair.

SOURCES

Competition: cyberchallenge.in/tcq2026. AlxCC: SoK on competition design, architectures and lessons learned (arXiv 2602.07666); DARPA results; ATLANTIS (arXiv 2509.14589); Trail of Bits Buttercup; Shellphish ARTIPHISHELL post-mortem; OpenSSF on the challenge legacy. Agents: OpenAI Aardvark/Codex Security; Google DeepMind CodeMender; Google Big Sleep. Research: Code-Augur specification inference (arXiv 2606.18619); LLMxCPG (USENIX Security 25); VulReaD (arXiv 2602.10787); Invalidator patch correctness via invariants (arXiv 2301.01113); Daikon; QASan; RapidPatch. Tooling: GitNexus documentation. Environment: BOSS Linux; Maya OS; Terrier Cyber Quest 2025 results.